Lerna Protocol

Hydra-native Virtual Channel Factories for Cardano eUTxO

Arthur Zhang

5 June 2026

Abstract

Lerna Protocol is a proposed design note for a virtual-channel factory discipline in Hydra-family Cardano eUTxO systems. It defines a Hydra-native method for separating stable value authority, moving signed state evidence, partitioned conservation, touched leaves, and local proof discipline. Lerna is not "eltoo for Cardano", not a replacement for Hydra, and not a Hydrozoa competitor. Its narrower claim is that a Head-like ledger can host compact rights and virtual channels, provided those rights remain deterministically unfoldable into ordinary ledger outputs.

The layer boundary is the paper's main point. Hydra virtualises Cardano execution inside one Head. Interhead virtualises Heads across Heads. Lerna virtualises rights and channels inside a physical Head, an Interhead virtual Head, or a sufficiently Head-like ledger. Hydrozoa decouples execution from custody and specifies deterministic outer rollout; Lerna borrows that design ethic as inner materialisation of compact factory rights into Head UTxOs before outer fanout or rollout. Compactness is therefore not free compression. It is a debt that must be repaid before the outer settlement mechanism is asked to finalise concrete outputs.

The concrete V0 target is deliberately modest: ADA-only, Head-local, and dependent on existing Hydra close, contest, and fanout semantics. V0 safety requires at least one authorised Head participant, operator, or delegated watcher to checkpoint or materialise the latest relevant Lerna state into a Hydra snapshot before contestation expires. If that assumption is unacceptable, the application must move to a future V1 claim-UTxO or another claim design; this note does not claim V1 feasibility. The originality claim is limited to three prototype obligations: C1, an inner factory-rights discipline; C2, deterministic inner materialisation before outer fanout or rollout; and C3, factory-local non-interference plus release receipts.

Keywords: Cardano, Hydra, Hydrozoa, eUTxO, virtual channels, channel factories, native assets, Aiken, Plutus, state channels, non-interference.

Contents

1 Introduction and Motivation

Hydra Head is the first practical member of the Hydra family. It lets a fixed set of participants run Cardano-like transactions off-chain with low latency, confirm state in signed snapshots, and return the resolved UTxO set to L1 through close, contest, and fanout. Its implementation path also includes incremental deposits, decommit-style exits, and partial fanout work. These features make Hydra a natural substrate for application-specific L2 protocols, but they expose a precise economic and operational question: must every bilateral channel, balance, rebalance, and local exit become a Head-visible UTxO while the application is still in its fast path?

Lerna answers no. It proposes a Head-local factory state that can hold many virtual-channel rights compactly while retaining deterministic paths back to ordinary Cardano outputs. The protocol is aimed at wallets, payment applications, app-specific liquidity groups, and decentralised applications that want the speed of Hydra, including Hydra Pay-like application workflows, but do not want to represent every temporary virtual channel as a full Head UTxO during normal operation [5].

The positioning is deliberately narrow: Hydra virtualises Cardano execution inside one Head; Interhead virtualises Heads across Heads; Lerna virtualises rights and channels inside a Head-like ledger. Compactness is useful only if it has a repayment rule. Before outer settlement, compact rights must be unfolded into Cardano-valid outputs, reabsorbed under fresh agreement, or deliberately released.

This work is motivated by the need for a Cardano-native virtual-channel factory that keeps stable funding, moving state evidence, operator publication duties, partitioned conservation, and factory-local proofs explicit inside the Hydra execution model. Cardano provides eUTxO, native assets, Plutus and Aiken validators, inline datums, reference inputs, reference scripts, min-ADA, transaction validity intervals, and Hydra's own Head state machine. Lerna's reviewable claims therefore stand on Cardano/Hydra semantics, formal V0 objects, and prototype evidence.

The practical problem is not only throughput. It is exit discipline. A virtual channel factory is useful only if the participants can tell, at every point, what would happen if the outer Head stopped making progress. Lerna therefore treats materialisation as a first-class protocol object. Before a Head fans out, Lerna V0 must expand every unresolved virtual right into ordinary Head UTxOs or agreed zero-value releases. Lerna V1 may instead fan out a compact claim object, but only if a real validator design can enforce latest-state settlement and non-interference on L1.

Research contributions and claim scope. This paper makes three Cardano-specific design contributions. They are stated as protocol obligations and prototype targets, not as finished cryptographic theorems.

C1. Inner factory rights discipline.
Lerna sketches compact rights for virtual channels inside a physical Hydra Head or an Interhead virtual Head. These rights are not miniature Heads and not L1 claims in V0. They are Head-local accounting claims over ADA balances, reserves, memberships, local exits, and settlement descriptors.

C2. Deterministic inner materialisation before outer fanout or rollout.
Lerna turns compact factory rights into ordinary Head UTxOs before Hydra fanout, or before an outer Hydrozoa-style rollout. Compactness is therefore not treated as free compression; it creates an explicit materialisation debt that the protocol must repay before the outer channel settles.

C3. Factory-local non-interference plus release receipts.
Lerna defines the proof obligation for reduced-signature local exits: touched leaves and a rights-dependency graph must show that untouched participants are

not affected. The ADA-only V0 prototype should use full factory authorisation unless this predicate is implemented and tested. Release receipts then remove materialised rights from compact state, preventing the same right from being claimed both as a factory entry and as a Head UTxO.

These contributions are not citation claims. They are protocol obligations that can be falsified by a prototype: if C1 cannot be represented as a compact Head-local state object, Lerna collapses into ordinary Head UTxO management; if C2 cannot be completed within the Head's liveness envelope, compactness is unsafe; if C3 cannot be checked without full-factory signatures in ordinary local exits, the first prototype should honestly remain a full-authorisation factory rather than claim reduced-signature security.

Equivalently, the originality claim is not that Lerna discovers state channels, virtual heads, deterministic rollout, or latest-state settlement. Those ideas already exist in the surrounding literature. The claim is that a Hydra-family ledger needs a smaller internal discipline: compact rights that are cheap while the Head is open, but that carry an explicit repayment rule before the outer ledger is asked to settle concrete UTxOs. This is the point at which the three contributions meet: C1 creates compact inner rights, C2 repays their fanout debt, and C3 prevents that repayment from double-spending or mutating untouched rights.

Remark 1 (ADA-only first claim)**.** *The only implementation-level claim made for V0 in this note is an ADA-only Head-local factory. Native assets are important to the Cardano design space, but they are treated here as an extension and benchmark agenda because Hydra's current known limitations around unburned native tokens and intrinsically large outputs can directly invalidate careless fanout plans.*

2 Why Not "Eltoo for Cardano"

The eltoo lineage is useful but narrow. It gives one important intuition: a newer mutually authorised state should supersede stale evidence without penalty. That intuition is not enough to define a Cardano protocol.

Lerna is not eltoo for three reasons. First, Hydra already has a dispute surface. A Head closes with a snapshot; participants may contest with a newer snapshot; after the contestation deadline, fanout distributes the resolved UTxO set. Lerna must fit into this close/contest/fanout mechanism rather than invent a parallel L1 replacement game for V0.

Secondly, Cardano's accounting is not Bitcoin's output scripting model. Even the ADA-only V0 path must account for lovelace, min-ADA, script datums, reference inputs, reference scripts, and validator resource limits; a later multi-asset profile must also account for native assets by policy ID and asset name. A "newer state wins" slogan does not define how a compact factory state materialises into valid eUTxO outputs.

Thirdly, the Cardano community already has Hydra-family research directions. Interhead Hydra studies composition across Heads. Hydrozoa explores a flexible multi-party state-channel design that decouples execution from custody and has a rule-based fallback regime. Lerna is positioned as an inner virtual-channel factory layer that can live inside these systems. It is not a replacement narrative for any of them.

Remark 2 (Positioning boundary)**.** *Hydra provides the outer L2 state channel and snapshot/fanout machinery. Hydrozoa-like systems may provide more flexible custody, dynamic membership, and fallback regimes. Lerna provides factory-local semantics for virtual channels, latest-state headers, release receipts, non-interference proofs, and materialisation plans.*

3 Related Work

3.1 Hydra Head

Hydra Head is an isomorphic multi-party state channel for Cardano [1]. Participants deposit UTxOs, run Cardano-like transactions off-chain, confirm state in snapshots, close with a snapshot if needed, contest stale snapshots, and fan out the resolved state back to L1 [2]. The implementation and documentation are directly relevant for Lerna because they define the outer lifecycle and the practical limits that Lerna must respect: participant limits, transaction size and execution-budget limits, deposit and decommit mechanics, snapshot numbering, contestation deadlines, and partial fanout [3].

The known limitations are especially important. Hydra documentation warns that transaction-size and execution-budget limits bound the number of participants and the final UTxO set that can be fanned out, and it still highlights two application-facing hazards: too many Head UTxOs can prevent finalisation, and tokens minted but not burned inside an open Head can also prevent finalisation [4]. Partial fanout and improved fanout costing can reduce batching pressure, but they do not make an intrinsically invalid, over-sized, or badly tokenised output safe. Lerna's compact factory state directly responds to this pressure: it avoids making every virtual channel a Head-visible UTxO during the fast path. The response is only safe because compact rights later undergo deterministic materialisation into valid Head UTxOs before outer settlement.

The closing consequence for Lerna is precise: Hydra is the outer settlement and fanout substrate, and Lerna V0 does not modify that substrate. Lerna behaves as a Head-local ADA application whose compact state is unfolded before fanout. Later profiles that use native assets, large inline datums, or mint/burn boundaries must prove compatibility with Hydra finalisation separately.

3.2 Interhead Hydra

Interhead Hydra studies virtual Hydra heads across two partially overlapping Hydra Heads [8]. Its construction operates an Interhead state machine in two Heads, lets intermediaries who belong to both Heads provide collateral, and can convert the virtual Head into a regular Hydra Head if the optimistic path fails. The paper is especially important for Lerna because it establishes a Cardano-native way to think about virtual Head-like state without opening that Head on L1 in the optimistic case.

The closing distinction is the layer boundary. Interhead virtualises a Hydra Head across Heads. Lerna virtualises rights and channels inside a physical Head or inside an Interhead virtual Head. Interhead's main hard problem is synchronising partial state machines and collateralising a virtual Head across intermediaries. Lerna's main hard problem is proving factory-local non-interference and completing deterministic materialisation before the containing Head-like ledger settles.

3.3 Hydrozoa

Hydrozoa is a Cardano multi-party state-channel project that explicitly separates execution from custody [7]. Its draft specification uses a multisig regime for the optimistic case, a rule-based Plutus regime for fallback, an L2 ledger of active UTxOs, minor/major/final blocks, deposits, withdrawals, treasury state, dispute votes, and deterministic rollout of withdrawn UTxOs that do not fit in one Cardano transaction. These choices are highly relevant to Lerna because they show a mature Cardano-specific design pattern: ordinary operation may be cheap and multisig-like, but the exit path must be deterministic, bounded by Cardano transaction limits, and explicit about which state version is being settled.

Lerna should use Hydrozoa cautiously. The local repository notes that Hydrozoa is pre-alpha and that public specifications are in flux [6]. Even so, three architectural lessons are stable

enough to use as design constraints: execution and custody should be separated; deposits and withdrawals need explicit deadlines and margins; and large output sets need deterministic rollout rather than ad hoc transaction construction. The closing consequence is not competition: Hydrozoa studies outer rollout and execution-custody separation, while Lerna borrows that design ethic as inner rollout and materialisation of compact factory rights.

3.4 Eltoo, Channel Factories, and Perun

Eltoo is relevant as an intellectual ancestor for non-punitive latest-state settlement [11]. Lerna adopts monotonic state numbers and rejects penalty-based settlement as the default. It does not adopt Bitcoin-specific sighash assumptions, transaction replacement mechanisms, or output templates.

Channel factories show why shared funding can support many off-chain relationships [12]. Perun and related virtual-channel work show the broader adjudication pattern: off-chain state becomes enforceable through a dispute or adjudication mechanism [13]. The closing boundary is that these works provide intellectual lineage only. Lerna does not take Bitcoin-specific assumptions, and it reserves heavyweight Cardano adjudication for the future V1 claim-UTxO mode, whose feasibility is not claimed here.

4 System Model

4.1 Cardano L1

Cardano L1 is the settlement layer. It supplies eUTxO accounting, native assets, Plutus and Aiken validators, reference inputs, inline datums, reference scripts, transaction validity intervals, and min-ADA requirements. Lerna assumes no Cardano consensus change [10].

4.2 Hydra Head

A Hydra Head is the outer L2 execution environment. Participants submit transactions to Hydra nodes; the Head state evolves through signed snapshots. Any participant can close with a snapshot; others may contest with newer valid snapshots before the contestation deadline; final fanout distributes the resolved Head UTxO set back to L1. Lerna V0 assumes an open Head and uses only this existing machinery.

4.3 Lerna Factory

A Lerna factory is a compact Head-local object representing many virtual channels and their rights. In V0 it is a Head-internal UTxO or state object controlled by a validator or by agreed Head-local transaction rules. It contains commitments to balances, subchannels, membership, reserve claims, release receipts, and materialisation descriptors.

4.4 Virtual Participants, Watchers, and Operators

Virtual-channel participants may be the same as Head participants, a subset of them, or users represented by Head participants. The distinction matters. If a virtual participant is not a Head participant, the design must state who can submit, observe, contest, or materialise on that participant's behalf. Lerna V0 therefore treats operator and watcher responsibilities as part of the deployment profile.

Hydra already requires participants to remain sufficiently online for safe contestation. Lerna adds inner obligations: tracking latest virtual-channel states, checking release receipts, detecting stale local exits, and preparing materialisation plans before Head closure. Operators may pay

ordinary Head-level transaction fees or manage application liquidity. They do not receive implicit authority over factory-owned balances.

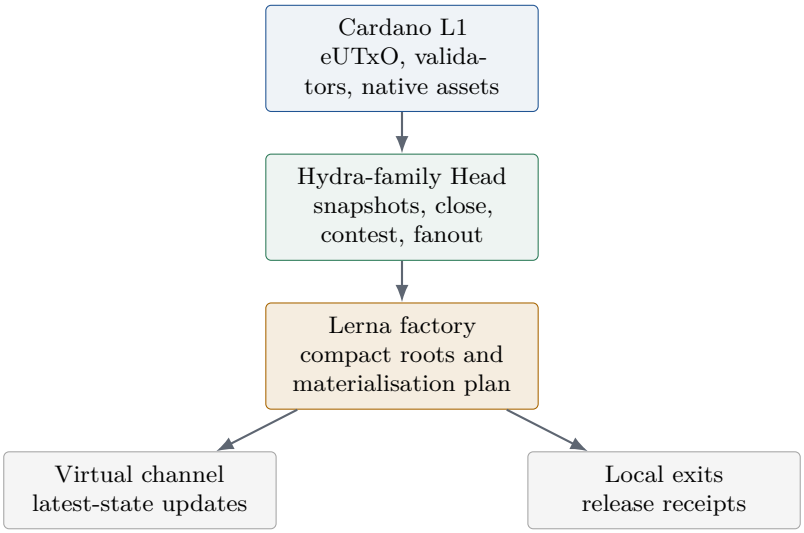

Figure 1: Lerna as an inner factory layer. Cardano L1 and Hydra supply the outer settlement and state-channel machinery; Lerna supplies compact factory-local semantics inside the Head.

5 Threat Model and V0 Enforcement Assumptions

Lerna V0 is conservative because it avoids a new L1 dispute protocol. That choice has a precise cost: a virtual participant does not receive an independent L1 exit path merely by participating in a Lerna virtual channel. In V0, the enforceable outer object is still the Hydra snapshot that survives close and contest. V0 safety therefore requires a concrete liveness path from the latest Lerna evidence to a Hydra snapshot before the contestation window closes.

5.1 Actors and Authority

Head participant. A Hydra party that can run a Hydra node, sign snapshots, submit Head-local transactions, close the Head, and contest with a newer snapshot.

Virtual participant. A user or application endpoint whose right is represented inside the Lerna factory. A virtual participant may also be a Head participant, but V0 does not assume this.

Operator. An application actor that may batch updates, fund ordinary transaction fees, or coordinate materialisation. An operator has no implicit authority to move participant balances unless the relevant transition signature set authorises that movement.

Watcher. An actor that observes Lerna latest-state evidence and Hydra close events. In V0 a watcher is useful only if it has a path to an authorised Head participant or is itself a Head participant. Observation without submission authority is not an enforcement path.

5.2 Adversary

The adversary may corrupt any subset of virtual participants, operators, watchers, and Head participants. It may withhold signatures, withhold proof material after signing a header, propose same-number conflicting states, refuse to include a valid Lerna checkpoint in a Head snapshot, close a Head with a stale snapshot, delay materialisation until the remaining contestation window is too short, and create materialisation pressure by opening many compact rights. It may also attempt ordinary replay across networks, Heads, factories, epochs, descriptor versions, and signature domains.

V0 does not assume an honest majority of Head participants. Its liveness condition is weaker and sharper: for every disputed right there must exist at least one non-corrupt actor with both the evidence and the authority path needed to get the newer Lerna checkpoint or materialised outputs into a newer Hydra snapshot before the contestation deadline. If every Head participant able to move the state forward is corrupt, censored, or offline, V0 offers no separate Lerna-level remedy.

5.3 Censorship Boundary

The Head-internal censorship attack is fundamental. A non-Head virtual participant may know the latest valid virtual state and still be unable to force it into a Hydra snapshot. In V0 that user's practical protection is delegation to a Head participant, an operator who is also a Head participant, or a watcher with a live submission path. This makes V0 a cooperative or watcher-assisted Head-local accounting layer. It is not a unilateral-exit protocol for arbitrary non-Head users.

Sharp V0 assumption block. V0 safety requires at least one authorised Head participant, operator, or delegated watcher to checkpoint or materialise the latest relevant Lerna state into a Hydra snapshot before contestation expires. This is not an availability optimisation; it is the enforcement bridge between compact inner rights and the outer close/contest/fanout substrate. If a deployment cannot make this bridge credible, V0 is the wrong mode and the design must move to V1 claim-UTxO research, with the validator feasibility burden made explicit.

Assumption 1 (Checkpoint-or-materialise liveness)**.** *For every latest Lerna state that may affect final rights, at least one authorised Head participant, operator, or delegated watcher can checkpoint that state or materialise its affected rights into the Hydra Head, and can ensure that the resulting Hydra snapshot is available before any relevant contestation deadline expires.*

Assumption 2 (Contestability)**.** *If a stale Hydra snapshot is used to close the Head, at least one honest watcher can observe the stale close, construct or obtain a newer Hydra snapshot containing the latest Lerna checkpoint or materialisation, and contest before the Hydra contestation deadline. The watcher may be a Head participant, an operator who is also a Head participant, a delegated watcher coordinating with a Head participant, or an application-specific watching service with an explicit submission path. V0 does not give a non-Head virtual participant an independent L1 claim.*

Who can act? The actor that moves latest Lerna evidence into a Hydra snapshot must be authorised at the Head boundary. In a minimal deployment this actor is a Head participant. In an application deployment it may be an operator who is also a Head participant, a delegated watcher coordinating with a Head participant, or an application-specific watching service with a clearly specified submission path. Observation alone is not enough: the deployment must identify the key, node, or service that can actually cause the newer snapshot or materialisation transaction to be proposed and signed. V0 does not specify a general delegation protocol for non-Head virtual participants; it requires the deployment to make that delegation explicit.

Assumption 3 (Materialisation capacity)**.** *The materialisation workload induced by the current factory state fits inside the deployment's transaction-size, execution-budget, min-ADA, native-asset, and contestation-time envelope. If this cannot be maintained with margin, the factory must enter shutdown-preparation mode and stop accepting new compact rights.*

Remark 3 (When V0 is not enough)**.** *If these assumptions are unacceptable for a target application, the application must move to V1 or another claim-UTxO design. V0 is a practical Hydra-local factory discipline, not a universal unilateral-exit protocol for users who are entirely outside the Head participant set. This paper does not present V1 as deployable.*

Remark 4 (Delegation open question)**.** *The unresolved V0 engineering question is how to encode watcher or operator delegation without making the operator a custodian of factory balances. A prototype should report the exact actor that can submit each checkpoint, materialisation, and contest transaction.*

6 Design Thesis

Hydra provides fast isomorphic eUTxO execution. Lerna provides virtual-channel factory discipline inside Hydra-family Heads.

The thesis has five parts.

T1. Compactness. Many virtual channels can be represented by one factory state object inside a Head, avoiding Head-internal UTxO explosion during normal operation.

T2. Deterministic unfoldability. Compact state must always admit a deterministic materialisation plan into ordinary Head UTxOs, because Hydra fanout ultimately distributes UTxOs rather than abstract rights.

T3. Latest-state settlement. Each virtual channel and factory transition uses monotonic state numbers and domain-separated signatures. Newer valid Lerna states supersede older Lerna states at the factory layer, without penalties.

T4. Non-interference. Reduced signing sets are safe only when touched leaves and dependency proofs establish that untouched participants' rights are unchanged.

T5. V0 before V1. The first practical implementation should keep all Lerna enforcement inside an open Hydra Head. L1 claim UTxOs are future research unless and until validators can enforce the necessary semantics within Cardano limits.

7 Lerna below Interhead: from virtual Heads to virtual rights

Interhead Hydra virtualises a Hydra Head across two partially overlapping Heads [8]. Its endpoints want something Head-like: an execution environment whose optimistic path can stay off-chain, whose intermediaries can provide collateral across the boundary, and whose failure path can convert the virtual Head into regular enforceable Hydra state. The object being virtualised is therefore a Head.

Lerna virtualises a smaller object. It does not create a virtual Hydra Head, does not add a cross-Head state machine, and does not inherit Interhead's conversion argument. A Lerna virtual channel is a right-bearing relationship inside a factory: a compact claim over balances, reserves, membership, local exit paths, and settlement descriptors. Its natural home is one physical Hydra Head, but the same factory state can in principle live inside an Interhead virtual Head. In that composition, Interhead supplies the virtual execution container and Lerna supplies the inner rights discipline.

The distinction can be summarised as:

Hydra Head	:	virtualises Cardano execution inside one Head,
Interhead	:	virtualises a Head across Heads,
Lerna	:	virtualises rights and channels inside a Head-like ledger.

This positioning changes the proof obligations. Interhead must synchronise partial state machines, intermediary collateral, and conversion phases. Lerna must show factory-local non-interference and deterministic materialisation. The two are compatible, but neither proof discharges the other. If a Lerna factory lives inside an Interhead virtual Head, timeout alignment becomes a first-class parameter: the inner materialisation schedule must complete early enough for the outer Interhead escalation and the eventual Hydra close, contest, and fanout path.

The interface between the two layers should therefore be narrow. Interhead need only see the Lerna factory as part of the virtual Head ledger state: a compact UTxO or state object whose transitions are authorised by that ledger. Lerna need only know that the containing ledger has a deterministic escalation path and a final UTxO-set semantics. Neither layer should depend on informal operator promises about how many outputs will be generated at settlement time.

This also explains why a Lerna channel should not be described as a smaller Interhead. Interhead preserves a Head-like enforceable state across two containers and, under dispute, can move that state into the ordinary Hydra state space. Lerna preserves rights inside one such container. Its escalation unit is not a virtual Head conversion; it is a deterministic materialisation batch, a receipt release, or a shutdown state that the container can include in its next authenticated state. The difference is small in wording but large in security accounting: Interhead reasons about Head membership, intermediary collateral, and conversion phases; Lerna reasons about right identifiers, touched dependencies, min-ADA reserves, and output descriptors.

Invariant 1 (Head-like ledger containment)**.** *A physical Head, an Interhead virtual Head, or a Hydrozoa-style ledger is a valid container for Lerna only if it gives Lerna three explicit interfaces: authenticated state inclusion, a bounded close or escalation path, and final settlement as ordinary UTxOs or UTxO-equivalent withdrawals. Lerna may rely on those interfaces, but the outer ledger is not required to interpret hidden factory rights. Any unresolved right that could affect final ownership must therefore be checkpointed, materialised, or receipt-released before the container finalises.*

8 Inner rollout before outer rollout: Lerna and Hydrozoa

Hydrozoa gives Lerna a second design reference because it separates execution from custody [7]. In Hydrozoa's optimistic multisig regime, the L1 script mostly sees a treasury controlled by unanimous peer signatures; the L2 consensus protocol interprets the ledger. If peers stop co-operating, the rule-based regime moves custody to Plutus scripts that resolve the latest compatible L2 block and then release funds according to that state. Hydrozoa's deterministic rollout algorithm exists because a large withdrawal set may not fit into one Cardano transaction.

Lerna borrows the ethic, not the full mechanism. Hydrozoa rolls an L2 ledger out to L1. Lerna rolls compact factory rights out to Head UTxOs before the outer system settles. This is inner rollout. It happens below Hydra fanout and below a Hydrozoa-style outer rollout:

compact Lerna rights
↓
ordinary Head UTxOs
↓
Hydra fanout or Hydrozoa-style L1 rollout

The ordering matters. If compact factory state is left unresolved until the outer channel is already closing, peers must negotiate the shape, order, min-ADA, datum policy, and native-asset

layout of many outputs at precisely the moment when liveness is weakest. Lerna therefore requires a deterministic materialisation plan while the Head is still open. V0 keeps that plan entirely Head-local. V1 would move some unresolved rights into a script-controlled claim UTxO, but that is a future research mode and not a deployable fallback in this paper.

The borrowed principle is custody clarity. In Hydrozoa, the optimistic ledger may execute cheaply while custody remains in a treasury whose rule-based fallback can be made explicit. In Lerna V0, virtual-channel updates may execute cheaply while custody remains in the containing Head UTxO set. Inner materialisation is the point where the compact right becomes a concrete output that the outer ledger already knows how to settle. This is why Lerna is Hydra-native rather than Hydrozoa-competitive: it uses the same conservative exit ethic at a lower layer.

The analogy must stop there. Hydrozoa's rule-based regime is an outer custody fallback: it moves control of the treasury to scripts that can resolve a compatible L2 block and roll withdrawals out. Lerna V0 has no such outer fallback of its own. Its compact factory datum is not a treasury and not a standalone custody object. It is only safe while the containing ledger can still include the materialisation transaction that turns compact rights into the ordinary outputs that Hydra or Hydrozoa already knows how to settle.

Proposition 1 (Inner-before-outer rollout)**.** *For V0, a compact Lerna right may remain compact only while the containing Head or Head-like ledger remains able to include a deterministic materialisation transaction in a later authenticated state. Once outer settlement is imminent, the right must be represented as an ordinary Head output, an agreed receipt release, or a full-participant shutdown state. Otherwise the outer rollout is being asked to settle a claim whose concrete Cardano output shape has not been agreed.*

Argument. Hydra fanout and Hydrozoa-style rollout settle UTxO sets or withdrawals, not application-private Merkle roots. If a compact right remains unresolved at the outer boundary, the parties still need to decide its address, value, min-ADA, datum, reference-script policy, and native-asset bundle. Those choices are consensus-relevant because different choices create different ledger outputs. V0 therefore treats inner materialisation as part of the state that must be settled before the outer mechanism is invoked. □

9 Why compact factory state matters for Hydra fanout

Hydra fanout distributes UTxOs, not abstract application rights. This is the central reason Lerna cannot stop at a compact Merkle root. A Head that closes with a large or awkward final UTxO set must still satisfy Cardano transaction size, execution-budget, native-asset, datum, and min-ADA limits. Hydra's known limitations make this operational rather than cosmetic: too many participants are bounded by configured and ledger limits; unburned native tokens minted inside a Head can prevent finalisation; partial fanout can leave problematic token-carrying outputs stuck; and an intrinsically oversized UTxO cannot be fanned out merely by splitting the fanout transaction [4]. The current documentation's warning that roughly dozens of Head UTxOs may already create fanout pressure should be read as an engineering signal rather than a hard protocol constant: the exact number depends on ledger parameters, output shape, scripts, datums, and implementation changes, but the direction of pressure is stable.

Compact factory state is Lerna's answer to fanout UTxO pressure. During normal operation, a factory may represent many virtual channels through a small number of Head-visible outputs. This reduces snapshot size, avoids unnecessary Head-local output churn, and keeps local rebalances from becoming fanout debt immediately. The answer is deliberately incomplete unless paired with deterministic materialisation. Compactness postpones UTxO expansion; it does not abolish it. Before Hydra fanout, every unresolved V0 right must either be materialised into an ordinary Head UTxO, absorbed into a fresh agreed factory state with no unilateral claim, or released by receipt.

This is why Lerna treats materialisation capacity as a safety assumption rather than an optimisation. A factory that admits more compact rights than it can unfold before the contestation deadline is unsafe even if every virtual-channel state is correctly signed. The protocol must measure output count, datum size, asset-bundle size, min-ADA reserve, and watcher response time as part of the factory policy.

Compactness therefore has a precise meaning. It is not a promise that the final Head state will be small. It is a promise that the fast path can keep temporary rights compact while the shutdown path remains capable of producing the ordinary Head UTxOs that Hydra requires. If that second promise is not measured and enforced, Lerna merely hides fanout work until the worst moment.

Invariant 2 (Fanout debt)**.** *Every compact active right carries a fanout debt equal to the deterministic Head-output work needed to settle it. A factory state is admissible only while the sum of its current fanout debt fits within the deployment's shutdown and contestation envelope, or while all affected parties have signed a cooperative state that removes any unilateral claim. This invariant is the price Lerna pays for avoiding Head-visible UTxO churn on the fast path.*

10 Compatibility with physical Heads, Interhead virtual Heads, and Hydrozoa-style ledgers

Lerna is a rights layer, so its compatibility question is not whether it replaces a channel protocol. The question is which outer ledger supplies execution, custody, and final settlement while Lerna supplies compact rights and inner materialisation.

Outer ledger	**Lerna role**	**Required boundary**
Physical Hydra Head	A compact factory UTxO or Head-local state object whose rights materialise before fanout	Uses Hydra close, contest, and fanout exactly as the outer dispute substrate; no V0 L1 dispute
Interhead virtual Head	A factory inside the virtual Head state, managing rights below the virtualised Head	Inner materialisation deadlines must fit inside Interhead conversion and outer Hydra contestation deadlines
Hydrozoa multisig-style ledger	A compact L2 ledger object whose withdrawals or local exits are deterministically unfolded	Peer consensus must checkpoint factory state and agree on the inner rollout plan before outer settlement
Hydrozoa rule-based-style ledger	A future claim-like object whose validator might adjudicate unresolved Lerna rights	V1 only; proof grammar, latest-state rule, and output rollout must be specified and benchmarked
Plain Cardano L1	A possible V1 claim UTxO after a Head has fanned out unresolved factory state	Future work; validator cost, proof size, native-asset handling, and min-ADA safety are open

The common rule is that Lerna must not smuggle new custody assumptions into the outer ledger. In V0, custody remains with the Hydra Head or with a Hydrozoa-style treasury. Lerna's authority is over compact factory rights and their Head-local materialisation.

Compatibility is therefore an adapter problem, not a replacement problem. The adapter binds four fields: the containing ledger identity, the state-inclusion rule, the escalation deadline, and the final output semantics. If any of those fields is informal, a Lerna deployment cannot claim V0 safety because the materialisation debt has no precise place to be repaid.

11 What Lerna deliberately does not take from each prior work

Lerna's originality depends as much on refusal as on borrowing.

From Interhead.
Lerna does not take Interhead's goal of virtualising an entire Hydra Head across Heads, nor its conversion machinery as a proof of Lerna safety. It takes the lesson that virtual state must have an explicit escalation path and that intermediary or operator resources must be visible.

From Hydrozoa.
Lerna does not take over Hydrozoa's treasury architecture, dynamic peer membership, or rule-based fallback as V0 machinery. It takes the separation of execution and custody, and the insistence that large settlement sets need deterministic rollout rather than ad hoc construction under stress.

From Hydra Head.
Lerna does not change Hydra's Head lifecycle, participation-token logic, close, contest, or fanout semantics. It takes Hydra as the outer substrate and accepts its practical constraints, including fanout UTxO pressure and native-token finalisation hazards. It also does not assume that partial fanout, decommit workflows, or future Hydra engineering improvements remove the need for Lerna's own deterministic materialisation plan.

From eltoo. Lerna does not take Bitcoin transaction replacement assumptions, sighash templates, or the slogan that latest state alone defines settlement. It takes only the non-punitive intuition that newer valid state should supersede stale evidence when the substrate can enforce that ordering.

12 Formal Objects

This section defines protocol objects at the design level. Encoding, CBOR schema, validator budget, and Aiken or Plutus implementation details are deliberately left for later specifications.

Definition 1 (LernaFactoryConfig)**.** *The factory configuration binds the factory to a Cardano network, a Head, a factory identity, a participant registry, and a materialisation policy.*

```
LernaFactoryConfig {
  protocol_version
  network_id
  hydra_head_id
  factory_id
  factory_epoch
  head_participant_set_hash
  virtual_participant_registry_root
  ledger_profile              // V0-ADA, multi-asset-extension, V1-claim
  asset_registry_root
  reserve_policy_hash
  fee_budget_policy_hash
```

```
  materialisation_policy_hash
  signature_scheme_id
  domain_separator
}
```

For the concrete V0 target, `ledger_profile = V0-ADA`. The factory validator or transaction rule must reject a factory input carrying non-ADA assets unless the profile is explicitly upgraded. This keeps the first security claim aligned with the prototype burden rather than with the full Cardano asset surface.

Definition 2 (FactoryState)**.** *The factory state is the compact state of the Lerna factory. It is not a Cardano transaction by itself. It is the object that Head-local transactions update and that materialisation plans unfold.*

```
FactoryState {
  config_hash
  factory_state_number
  balances_root
  subchannels_root
  membership_root
  reserve_root
  release_receipt_root
  pending_materialisation_root
  settlement_descriptor_hash
  asset_registry_hash
  operator_budget_state
}
```

Definition 3 (VirtualChannelState)**.** *The virtual-channel state is the latest-state object for one virtual channel. In a payment-only prototype,* `app_state_hash` *may be empty and balances are enough. In a dApp channel, it commits to application-specific eUTxO or state-machine material.*

```
VirtualChannelState {
  virtual_channel_id
  factory_id
  participants
  virtual_state_number
  balances
  locked_reserve
  app_state_hash
  settlement_descriptor_hash
  expiry_or_liveness_profile
}
```

Definition 4 (LatestStateHeader)**.** *The header is the signed authority object. It must be domain-separated from ordinary Cardano transactions, Hydra snapshots, wallet messages, and earlier protocol versions.*

```
LatestStateHeader {
  domain_separator
  protocol_version
  network_id
  hydra_head_id
  factory_id
  factory_epoch
  state_scope
  state_number
```

```
  previous_state_hash
  new_state_hash
  transition_family
  touched_root
  settlement_descriptor_hash
  materialisation_plan_hash
  validity_interval
  hydra_snapshot_hint
}
```

Signature domain. The V0 signature scheme is Ed25519 over a canonical CBOR encoding of the `LatestStateHeader`; the digest is Blake2b-256 of the domain-tagged payload. The initial domain tags are:

`lerna:v0:latest-state-header`	state supersession,
`lerna:v0:release-receipt`	right release and anti-replay,
`lerna:v0:materialisation-plan`	batch ordering and output plan,
`lerna:v0:watcher-delegation`	optional delegation evidence.

The signer registry must state whether a key is a Hydra party key, a Cardano payment/stake key used for off-chain messages, or an application key. These key roles are not interchangeable.

Definition 5 (AccessManifest and TouchedLeaves)**.** *The access manifest is the envelope-first admission object. It declares what kind of transition is being attempted before proof bytes are interpreted.*

```
AccessManifest {
  transition_family
  authorisation_policy_id
  affected_participants
  required_signers
  read_set_root
  write_set_root
  inserted_set_root
  removed_set_root
  dependency_set_root
  asset_delta_summary
  reserve_delta_summary
}

TouchedLeaves {
  balance_leaves
  subchannel_leaves
  membership_leaves
  reserve_leaves
  release_receipt_leaves
  absence_proofs
  membership_proofs
}
```

Definition 6 (NonInterferenceProof)**.** *The proof must establish that all rights outside the transition footprint remain unchanged, including indirect rights through shared reserve, membership, exit paths, and sponsor or operator budgets. A Merkle proof alone is not a non-interference proof.*

```
NonInterferenceProof {
  access_manifest_hash
  rights_dependency_graph_hash
  untouched_rights_commitment
  unchanged_dependency_paths
  conservation_witness
}
```

Definition 7 (SettlementDescriptor)**.** *The descriptor is not an application runtime. It is a bounded rule set for deriving Cardano outputs or Head-local outputs from Lerna rights.*

```
SettlementDescriptor {
  descriptor_version
  output_derivation_rules
  participant_destination_policies
  asset_allocation_rules
  min_ada_rules
  datum_and_reference_script_rules
  local_exit_rules
  rebalance_and_splice_rules
  fanout_preparation_rules
}
```

Definition 8 (FanoutMaterialisationPlan)**.** *In V0,* `claim_utxo_descriptor_hash` *must be empty because unresolved factory state is not allowed to survive Head fanout. In V1 it may describe a script-controlled L1 claim UTxO.*

```
FanoutMaterialisationPlan {
  plan_id
  source_factory_state_hash
  materialisation_mode
  output_set_hash
  claim_utxo_descriptor_hash
  unresolved_channel_set_root
  batch_index
  batch_count
  budget_profile_hash
  required_signatures
  watcher_deadline_profile
}
```

Definition 9 (ReleaseReceipt)**.** *Release receipts prevent a common ambiguity in compact factories: after a virtual right is materialised into an ordinary Head UTxO, the compact factory must no longer carry a spendable copy of that right.*

```
ReleaseReceipt {
  receipt_id
  receipt_domain_separator
  released_right_id
  factory_id
  factory_epoch
  virtual_channel_id
  state_number
  right_commitment_hash
  materialisation_plan_hash
  batch_index
```

```
  output_index
  replacement_output_hash
  consumed_in_factory_state_hash
  signer_set_hash
  signature
}
```

Definition 10 (Release receipt anti-replay rule)**.** *Let*

$$\begin{aligned}\mathsf{receiptId} = \mathsf{H}(&\textit{lerna:v0:release-receipt}, \mathsf{networkId}, \mathsf{hydraHeadId},\\ &\mathsf{factoryId}, \mathsf{factoryEpoch}, \mathsf{releasedRightId}, \mathsf{stateNumber},\\ &\mathsf{materialisationPlanHash}, \mathsf{batchIndex}, \mathsf{outputIndex}).\end{aligned}$$

A receipt is valid only if this identifier is absent from the old release_receipt_root*, present in the new* release_receipt_root*, the released right is absent from the new active-right commitments, and the referenced materialised output hashes exactly to* replacementOutputHash*. The receipt signature uses the* `lerna:v0:release-receipt` *domain and the same signer policy that authorised the materialisation. A receipt cannot be replayed across Heads, factory epochs, batch positions, or replacement outputs.*

13 Head-local Validator Sketch

The first implementation target should be an ADA-only Head-local factory validator, or a validator-compatible transaction rule used by a Hydra application. This does not modify Hydra protocol internals. The following is intentionally Aiken-like pseudocode rather than production Aiken [9]. Its purpose is to show the V0 prototype direction and the minimum checks that belong on the compact factory UTxO or equivalent Head-local rule.

```
validator lerna_factory {
  spend(datum: FactoryDatum, redeemer: FactoryRedeemer, ctx: ScriptContext) {
    let old = datum.state
    let new = redeemer.new_state
    let manifest = redeemer.access_manifest
    let touched = redeemer.touched_leaves
    let receipts = redeemer.release_receipts

    check_factory_identity(old.config_hash, new.config_hash)
    check_head_binding(old.hydra_head_id, ctx)
    check_state_number_increases(old.state_number, new.state_number)
    check_ledger_profile_is_ada_only(old, new, ctx)

    check_domain_separated_signatures(
      redeemer.latest_state_header,
      manifest.required_signers,
      redeemer.signatures,
    )

    check_manifest_commits_to_body(manifest, redeemer.proof_body)
    check_touched_roots_recompute(old, new, manifest, touched)

    if manifest.required_signers != all_factory_participants {
      reject_unless_non_interference_profile_is_active()
      check_non_interference(manifest, touched, redeemer.non_interference_proof)
    }
```

```
    check_release_receipts_are_fresh_and_consuming(old, new, receipts)
    check_ada_conservation(old, new, redeemer.materialised_outputs)
    check_min_ada_for_materialised_outputs(redeemer.materialised_outputs)
    check_operator_budget_boundary(old, new, manifest)
  }
}
```

This sketch is deliberately narrow. It is not a Hydra protocol modification and does not claim that the V1 L1 claim validator is feasible. It only identifies the V0-ADA checks that a prototype should measure inside a Hydra-local transaction: identity, monotonicity, signature domain, access-manifest binding, root recomputation, full-authorisation fallback, release-receipt consumption, ADA conservation, min-ADA constraints, and operator-budget separation. Native asset conservation remains an extension check, not part of the first V0 claim.

14 Minimal ADA-only V0 State Machine

The V0 prototype should be specified as a small state machine before any multi-asset or V1 claim machinery is attempted. A factory state carries:

$$F = (c, n, m, B, R, Q, E, \rho, \pi, \omega),$$

where c is the configuration hash, n the factory state number, m the mode, B the balance/right commitments, R the reserve commitments, Q the pending materialisation commitment, E the release-receipt commitment, ρ the participant registry commitment, π the settlement descriptor hash, and ω the operator-budget state. In V0-ADA the value lanes are lovelace only, split into business balances, min-ADA reserve, shared reserve, and operator budget.

$$m \in \{\mathsf{Init}, \mathsf{Open}, \mathsf{Checkpointing}, \mathsf{Materialising}, \mathsf{ShutdownPreparing}, \mathsf{Settled}, \mathsf{Aborted}\}.$$

Transition	**Mode change**	**Admissibility summary**
initFactory	Init → Open	Head id, factory id, descriptor, signer registry, and ADA-only ledger profile are bound; initial ADA conservation holds.
openVC	Open → Open	Required signers authorise a new virtual right; reserve and min-ADA accounting increase consistently; no native asset is admitted in V0-ADA.
checkpoint	Open → Checkpointing → Open	A domain-separated latest-state header strictly increases the relevant state number and is included in a Hydra snapshot.
rebalance	Open → Open	The access manifest declares both source and destination rights; ADA is conserved across affected rights and reserve lanes.
localExit	Open → Materialising	Latest evidence, touched leaves, and either full factory authorisation or an active non-interference proof authorise materialisation of selected rights.

Transition	Mode change	Admissibility summary
materialiseBatch	Materialising → Materialising or Open	The canonical batch creates ordinary Head outputs, writes fresh release receipts, removes active compact rights, and updates the pending suffix commitment.
prepareShutdown	Open → ShutdownPreparing	Materialisation debt exceeds the configured threshold or the Head is expected to close; new compact opens stop.
cooperativeSettle	Open or shutdown-preparing to settled	All remaining rights are materialised, released, or covered by full-participant shutdown signatures; no unresolved unilateral claim remains.
abortV0	any non-settled mode → Aborted	Entered only as an analytical state when the liveness bridge or materialisation capacity assumption is violated; V0 specifies no independent recovery from it.

Definition 11 (Transition validity)**.** ValidV0ADA(F, F', T, W) *holds when the transition* T *and witness* W *satisfy factory identity, Head binding, strict state-number monotonicity, signature-domain correctness, manifest-to-body binding, touched-root recomputation, release-receipt freshness, ADA conservation, min-ADA reserve sufficiency for every materialised output, and the operator-budget boundary. If* T *uses fewer than all factory signers, validity additionally requires an active and benchmarked* ValidNonInterference *predicate; otherwise the transition is invalid.*

Remark 5 (Same-number equivocation)**.** *For a fixed* (network, head, factory, epoch, scope)*, two different headers with the same* stateNumber *and different* newStateHash *are an equivocation event. V0 does not try to pick a winner. The affected scope must freeze until the parties sign a strictly higher repair state or the factory falls back to full-participant shutdown. This avoids silently accepting "highest number wins" when the number is tied.*

15 Protocol Lifecycle

15.1 Open Head

Participants open a Hydra Head under the ordinary Hydra lifecycle. In the current practical Hydra workflow, Lerna should treat Head opening, deposit, and decommit as outer protocol operations rather than as Lerna-specific actions. Lerna does not modify this process.

15.2 Initialise Lerna Factory

The Head participants create an initial Lerna factory state inside the open Head. A conservative prototype may represent it as one script-controlled Head UTxO with an inline datum containing the `FactoryState` hash and enough data to update it. More compact implementations may store only the roots and keep proofs off-chain.

Initialisation must bind the Cardano network id, Hydra Head id, Lerna factory id, participant registry root, asset registry root, settlement descriptor hash, and materialisation policy.

15.3 Open Virtual Channel

Opening a virtual channel updates the factory state. The transition touches membership, reserve, balances, and subchannel roots. It requires signatures from the virtual-channel participants and any factory participants whose reserve or exit rights are affected. If the opening changes shared reserve or membership in a way that affects everyone, full factory authorisation is required unless non-interference proves otherwise.

15.4 Off-chain or Head-local Update

There are two update styles. In the first, virtual-channel participants exchange signed latest-state headers off-chain and only submit to the Head when they need to checkpoint, rebalance, exit, or respond to conflict. In the second, a compact factory UTxO is updated by a transaction inside the Head, and the resulting Head snapshot records the new factory roots.

The V0 prototype should support Head-local checkpointing first, then add purely off-chain virtual updates once materialisation and release receipts are well tested.

15.5 Local Rebalance, Splice, and Exit

A rebalance moves value between virtual channels without changing the outer Head deposit. A splice changes the compact factory reserve by depositing into, decommitting from, or rearranging Head-local value. Lerna must distinguish these carefully: rebalance changes rights inside the factory; splice changes the factory's Head-visible value or materialisation reserve; neither may silently change the factory id or replay domain.

A local exit lets a participant materialise a virtual-channel right without closing the entire factory. It requires latest-state evidence for the relevant virtual channel, release receipts for rights removed from compact state, touched leaves and membership proofs, non-interference proof for untouched participants or full factory authorisation, and exact output materialisation under the settlement descriptor.

15.6 Materialise Inside Head

Materialisation converts compact factory rights into ordinary Head UTxOs. In V0 this is the main safety operation. Before Head close, every unresolved virtual channel must either materialise into ordinary Head outputs, be reabsorbed into the factory with fresh signatures, be closed with release receipts, or be covered by a full-participant shutdown plan.

The materialised outputs must satisfy Cardano eUTxO constraints, including min-ADA and native-asset accounting.

15.7 Head Close, Contest, and Fanout

If the Head is closed, Hydra's own close and contest rules decide which Head snapshot is final. Lerna V0 does not create a separate L1 dispute. Therefore, Lerna participants must ensure the final Head snapshot contains either ordinary materialised outputs or a fully agreed factory shutdown state that has no unresolved unilateral claims.

If a stale Head snapshot omits a newer Lerna materialisation, the answer is not a Lerna-specific L1 action. The answer is to contest the Head with a newer Hydra snapshot that includes the correct Lerna state or materialised outputs. This is why watcher responsibilities must cover both Hydra snapshots and Lerna latest-state evidence.

15.8 V1 Claim-UTxO Fanout

V1 is future work. If the Head closes before all virtual channels are materialised, the final Head UTxO set may include a script-controlled L1 claim UTxO representing the unresolved factory

state. An Aiken or Plutus validator would then enforce domain-separated latest-state signatures, state monotonicity, local exits, non-interference proofs, native-asset conservation, min-ADA-safe output creation, and release receipt consumption.

This is research-heavy. Until such validators are specified and tested, V1 should not be presented as deployable.

16 Latest-state Settlement Semantics

Lerna uses monotonic state numbers. For a fixed

$$(\mathsf{network}, \mathsf{head}, \mathsf{factory}, \mathsf{epoch}, \mathsf{scope}),$$

a valid state with number $n + k$, $k > 0$, supersedes a valid state with number n at the Lerna layer if the header is domain-separated, signatures match the required signing set, the transition family is admissible, touched leaves recompute the old and new roots, conservation checks pass, and release receipts prevent double materialisation.

There is no penalty mechanism. A participant presenting stale evidence does not lose funds by punishment. The safety objective is simply that stale evidence cannot finalise if newer valid evidence is available through the appropriate Hydra contest or Lerna materialisation path.

Lemma 1 (Hydra inclusion lemma for V0)**.** *Assume an honest authorised actor observes a newer valid Lerna state S' before the Hydra contestation deadline, can construct a Head transaction that checkpoints or materialises S', obtains a Hydra snapshot containing that transaction, and successfully contests a stale close with that newer snapshot. Then the final Hydra fanout follows the newer snapshot rather than the stale snapshot.*

Argument. This is a restatement of the outer Hydra close/contest mechanism applied to a Head state that happens to contain Lerna data. In V0, Lerna has no independent L1 dispute mechanism. Once S' is included in a newer valid Hydra snapshot, Hydra's contest rule is the mechanism that prevents the older snapshot from being final. The lemma proves only the inclusion bridge; it does not prove that a non-Head virtual participant can force inclusion, that proof material will be available, or that materialisation will fit in the remaining window. □

Remark 6 (Not a Lerna security theorem)**.** *The lemma should not be cited as a standalone Lerna latest-state safety theorem. The hard Lerna obligations are exactly the premises: authorised inclusion, proof availability, materialisation capacity, receipt consumption, and timely contest. A deployable claim would need to make those premises mechanisms rather than assumptions.*

17 Factory-local Proof Mode

A Lerna factory may contain many balances and subchannels. Requiring all participants to sign every local action is safe but undermines the purpose of a factory. Requiring only affected participants to sign is unsafe unless the protocol proves that untouched participants are not affected.

The proof mode has four layers:

(1) commitment roots for balances, subchannels, membership, reserve, and release receipts;

(2) an access manifest declaring the transition family and footprint before proof interpretation;

(3) touched leaves for the items read, written, inserted, removed, or proven absent;

(4) a non-interference proof showing that untouched rights remain unchanged, including dependency paths through shared reserve and membership.

Definition 12 (Rights dependency graph)**.** *For a factory state F, the rights dependency graph is a labelled directed graph*

$$G_F = (V, E, \lambda)$$

whose vertices are right-bearing atoms:

$$V = V_{\mathsf{balance}} \cup V_{\mathsf{reserve}} \cup V_{\mathsf{membership}} \cup V_{\mathsf{descriptor}} \cup V_{\mathsf{receipt}} \cup V_{\mathsf{operator}}.$$

An edge $u \to v$ means that changing u may change the value, exit path, materialisation viability, or authorisation condition of v. For example, a shared reserve vertex points to every balance right that depends on that reserve; a membership vertex points to the signing policy and local-exit rights that use that member set; a descriptor vertex points to every right whose materialised output it derives. The graph commitment is part of the factory state, so a transition cannot choose its dependency graph after seeing which proof is convenient.

Definition 13 (Touched closure)**.** *For a transition with access manifest M, let $A(M)$ be the set of vertices declared read, written, inserted, removed, or proven absent. The touched closure is*

$$\mathsf{cl}_{G_F}(M) = A(M) \cup \{v \mid \exists u \in A(M) \text{ and a path } u \rightsquigarrow v \text{ in } G_F\}.$$

Reduced authorisation is admissible only for participants whose rights are inside this closure. Rights outside the closure must have unchanged membership, balance, reserve, descriptor, and exit-path commitments.

Definition 14 (Valid non-interference predicate)**.** $\mathsf{ValidNonInterference}(F, F', M, W)$ *holds only if W proves:*

(a) the committed dependency graph used for F and F' is the graph declared by M;

(b) all writes, insertions, removals, and absence claims are contained in $\mathsf{cl}_{G_F}(M)$;

(c) every vertex outside $\mathsf{cl}_{G_F}(M)$ has the same committed right in F and F';

(d) ADA conservation holds over the closure plus any authorised splice, operator-budget, or materialised-output lanes;

(e) the reduced signer set covers every participant whose right lies inside the closure or whose authorisation policy is changed by the transition.

If any of these checks cannot be represented by the chosen proof system, V0 must reject the reduced-signature transition and require full factory authorisation.

Invariant 3 (Envelope-first admissibility)**.** *A proof body is interpreted only after the access manifest has fixed the transition family and committed to that body. The proof bytes do not get to choose their own authority class.*

Invariant 4 (Locality is a proof obligation)**.** *If non-interference cannot be established for a reduced-signature transition, the transition falls back to full factory authorisation.*

18 Conservation Model

The first V0 ledger profile is ADA-only. Native assets are described here only as an extension target. This downgrade matters: Cardano native assets are powerful, but Hydra's known limitations around unburned tokens and large output bundles make native-asset fanout an unsafe first claim.

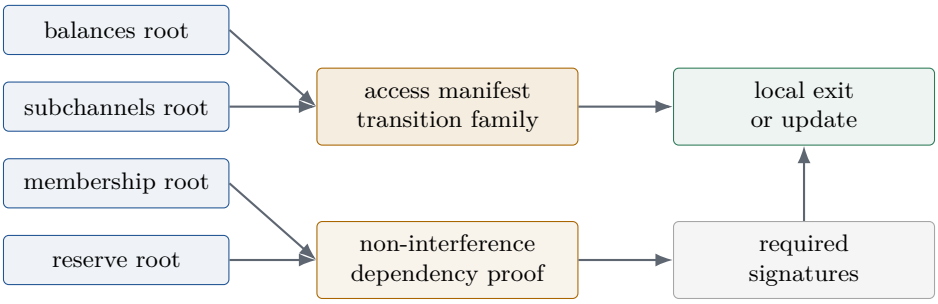

Figure 2: Factory proof mode. The touched footprint and the non-interference argument are different objects. A balance proof is not enough when reserve, membership, or exit rights depend on that balance.

18.1 ADA-only V0 Equations

For a factory state F, define:

$B(F)$	sum of spendable lovelace owed by active virtual rights,
$M(F)$	sum of lovelace reserved only for min-ADA of materialised outputs,
$R(F)$	shared factory reserve not assigned to a single virtual balance,
$O(F)$	operator budget authorised for fees and publication costs,
$P(F)$	pending materialised-output lovelace committed by the current plan.

For an ordinary internal update with no Head-visible splice and no materialisation,

$$B(F) + M(F) + R(F) + O(F) + P(F) = B(F') + M(F') + R(F') + O(F') + P(F').$$

For a materialisation batch producing ordinary Head outputs Y, let $\mathrm{ada}(Y)$ be the lovelace in those outputs and let fee_O be the operator-budget amount explicitly consumed by the transition. The admissibility equation is:

$$B(F)+M(F)+R(F)+O(F)+P(F) = B(F')+M(F')+R(F')+O(F')+P(F')+\mathrm{ada}(Y)+\mathrm{fee}_O.$$

Every output $y \in Y$ must satisfy the settlement descriptor and its min-ADA requirement; the min-ADA lane is not spendable business balance. A local exit that pays a participant by stealing from $M(F)$, $R(F)$, or $O(F)$ is invalid unless the transition family and signer set explicitly authorise the corresponding lane movement.

Invariant 5 (ADA-only conservation)**.** *For every admitted V0-ADA transition, the equations above hold, every created release receipt consumes exactly one active right, and the same right is not present in both $B(F')$ and a materialised Head output.*

18.2 Native-asset Extension

Cardano native assets are identified by policy ID and asset name. A future multi-asset Lerna profile must check a separate conservation equation for each asset lane

$$a = (\mathsf{policyId}, \mathsf{assetName}), \qquad Q_a(F) + I_a - M_a - Burn_a = Q_a(F') + Y_a,$$

where Q_a is the quantity committed in compact active rights and reserves, I_a is an explicitly authorised Head-local deposit or splice input, M_a and $Burn_a$ are authorised mint/burn quantities under the relevant policy, and Y_a is the quantity emitted into materialised outputs. Metadata, ticker symbols, and display names are not asset identity. This equation is not part of the ADA-only V0 prototype claim.

The following table contrasts ordinary accounting intuition with Lerna's required treatment.

Lane	What Lerna tracks	Main hazard
Lovelace balance	Spendable ADA owed to virtual participants	Accidentally using business balance to pay operator costs
Min-ADA reserve	Lovelace required to make materialised outputs valid	Double-counting output viability reserve as spendable balance
Native asset	Policy ID, asset name, and quantity	Treating ticker, metadata, or display name as asset identity
Factory reserve	Shared liquidity and exit reserve claims	Local transition affecting untouched participants through shared reserve
Operator budget	Fees and operational close/materialisation costs	Hidden leakage from virtual-channel balances
Release receipts	Rights already materialised or closed	Double claim through compact and materialised forms

Lerna's partitioned-conservation rule is a native Cardano accounting discipline: Cardano fees, min-ADA, native-asset bundles, operator budgets, and participant balances must be tracked separately. Publication or operation costs must not invisibly leak from participant balances.

19 Deterministic Materialisation

Hydrozoa's deterministic rollout algorithm is a useful warning: a state-channel paper that does not specify how large withdrawal sets become Cardano transactions has not finished its exit design. Lerna adopts the same design ethic but applies it one layer inward. The question is not only how a Head fans out; it is how compact factory rights become a Head UTxO set before that fanout.

Materialisation must be measurable. A V0 prototype should report, for a chosen Hydra contestation period, the number of rights unfolded, output count, datum size, min-ADA footprint, native-asset bundle size, transaction count, witness size estimate, snapshot-confirmation latency, and observed time-to-materialise. Without those measurements, compactness is only a presentation choice, not a safety argument.

This measurement is not a benchmark afterthought. It is part of the admission rule for opening new compact rights. A deployment that cannot estimate the remaining materialisation workload should refuse new virtual-channel opens and enter shutdown preparation before the Head is under close pressure. In particular, a factory that admits more compact rights than it can unfold within its chosen shutdown envelope is unsafe even when all relevant signatures are valid.

Definition 15 (Materialisation equivalence)**.** *A materialised Head output is equivalent to a compact Lerna right if its address, value, datum policy, reference-script policy, and release receipt match the settlement descriptor for that right. Equal value alone is insufficient.*

19.1 V0 Algorithm

Given a source factory state F, a settlement descriptor D, and a set of rights R to unfold, V0 materialisation proceeds under a committed $\mathsf{BudgetProfile}$. The profile contains conservative constants:

$$\mathsf{BudgetProfile} = (B_{\mathsf{bytes}}, B_{\mathsf{ex}}, B_{\mathsf{outputs}}, B_{\mathsf{datum}}, B_{\mathsf{value}}, B_{\mathsf{margin}}, S_{\mathsf{witness}}),$$

where the final component is a fixed witness-size estimate for the selected signing policy. This profile is part of the factory configuration or the materialisation policy hash, so peers compute the same batches offline.

(1) Sort R by canonical right identifier:

$$(\mathsf{factoryId}, \mathsf{virtualChannelId}, \mathsf{rightKind}, \mathsf{participantId}, \mathsf{nonce}).$$

(2) For each right, derive a candidate Head output using D. The derived output includes address, lovelace, datum, optional reference-script expectation, and min-ADA reserve. Under V0-ADA, reject the plan if any candidate output contains non-ADA assets.

(3) Reject the plan if any output fails Cardano output validity, min-ADA, or value-bundle constraints.

(4) Attach one release receipt per materialised right and update the release_receipt_root.

(5) Define

$$\mathsf{Fits}_P(k) = \mathsf{true}$$

exactly when the first k candidate outputs, receipts, datum updates, and fixed witness estimate fit within the budget profile P; otherwise $\mathsf{Fits}_P(k) = \mathsf{false}$. The batch size is

$$k^\star = \max\{k \mid 0 < k \leq |R|,\ \mathsf{Fits}_P(k)\}.$$

If no such k exists, the plan is invalid and the factory must enter shutdown-preparation or full-authorisation recovery.

(6) Materialise exactly the first $k^\star$ candidate outputs. The new factory state commits to the untouched suffix $R[k^\star + 1..]$ in pending_materialisation_root, increments batchIndex, and records the fresh release receipts for the prefix.

(7) Repeat deterministically until the suffix is empty or the shutdown deadline makes further cooperative progress unsafe.

Batch failure semantics. Only a confirmed Head-local transaction changes the factory state. If an off-chain preflight, signature collection, or transaction submission for batch i fails, the last confirmed factory state remains authoritative and the same batch can be reconstructed from $(F, D, R, \mathsf{BudgetProfile}, i)$. If batches $0..i-1$ have already been confirmed and the Head closes before batch i is confirmed, the pending suffix remains compact. In V0 that suffix has no independent L1 claim; the deployment is safe only if a newer Hydra snapshot including further materialisation can still be contested, or if all remaining rights are covered by full-participant shutdown signatures.

Concurrency rule. A factory may have at most one active materialisation plan for a given (factoryId, factoryEpoch, sourceFactoryStateHash). The planId, batchIndex, and release-receipt ids make concurrent duplicate plans conflict rather than silently create two claims for the same right.

This is not identical to Hydrozoa rollout. Hydrozoa rolls L2 withdrawals out to L1 through settlement/finalisation and rollout UTxOs. Lerna V0 rolls compact factory rights out to ordinary Head UTxOs before Hydra fanout. The shared idea is determinism: peers should not have to negotiate transaction minutiae at the moment of exit.

19.2 Shutdown Preparation

A Lerna deployment should define a shutdown-preparation threshold Θ_{mat}. If the expected materialisation work cannot be completed with margin before the Hydra contestation deadline, the factory must stop accepting new compact virtual-channel opens and begin unfolding rights. The threshold depends on:

$$\Theta_{\mathsf{mat}} = f(N_{\mathsf{rights}}, S_{\mathsf{datum}}, S_{\mathsf{assets}}, T_{\mathsf{head}}, T_{\mathsf{watch}}, B_{\mathsf{tx}}),$$

where N_{rights} is the number of rights to unfold, S_{datum} is the expected datum footprint, S_{assets} is the native-asset bundle footprint, T_{head} is the Head contestation period, T_{watch} is watcher response time, and B_{tx} is the transaction budget profile. This paper does not know f in advance and does not pretend that one symbolic function settles the engineering problem. It requires each deployment to measure a conservative bound for its own ledger profile. The safety question is whether the current compact state can be unfolded with margin, not whether its signatures are well formed in isolation.

19.3 V1 Claim-UTxO Discipline

V1 does not remove the need for deterministic materialisation. It moves part of the process to L1. A claim UTxO must still define a deterministic withdrawal order, a proof grammar, a state-number rule, and a way to split large output sets across multiple transactions. Otherwise V1 merely postpones the exit problem until the worst possible moment.

20 Hydra L2/L3 Interaction

Lerna is naturally an inner layer. At L1, Cardano provides settlement, validators, native assets, and finality. At L2, Hydra or a Hydrozoa-style Head provides fast shared execution and snapshot/fanout safety. Inside the Head, Lerna provides virtual-channel factory semantics. Above Lerna, applications can run payment channels, app-specific channels, local rebalances, and conditional exits.

This can be described as L3 only if the term is used carefully. Lerna is not a new global consensus layer above Hydra. It is a factory discipline whose ordinary state transitions can be off-chain relative to the Head and whose checkpoints can be Head-local.

Compatibility with Interhead and virtual Head directions is plausible but not solved. A virtual channel could connect liquidity positions represented in two Heads, but then Lerna needs cross-Head evidence, timeout alignment, and liquidity-discovery logic. Those are out of scope.

Hydrozoa dynamic membership may make Lerna more useful, not less. If Head membership can evolve, the factory participant registry and rights-dependency graph become even more important. Lerna should not assume static Head membership forever, but V0 should begin with a fixed participant set.

21 Security Properties

This section states target safety properties and the mechanism expected to enforce each one. The term “safety property” is used only where an enforcement mechanism is stated.

Anti-replay. State signatures cannot be replayed across networks, Heads, factories, factory epochs, state scopes, or descriptor versions because `LatestStateHeader` binds those fields under a domain separator. Assumptions: signature unforgeability, canonical encoding, and no domain-separator collision.

State monotonicity.
For a given state scope, valid transitions strictly increase `state_number`. V0 enforces this through Head-local factory update rules and by requiring any contested Head snapshot to include the latest valid Lerna checkpoint or materialisation.

Virtual-channel balance conservation.
In V0-ADA, lovelace balances are conserved per virtual channel unless a transition explicitly moves value through a declared rebalance, splice, deposit, exit, or release. Mechanism: access manifest, touched leaves, reserve delta checks, settlement descriptor, and the ADA equations in the conservation model.

Factory non-interference.
Untouched participants' rights remain unchanged when a reduced signing set authorises a local action. Mechanism: rights-dependency graph, non-interference proof, and fallback to full authorisation when proof is insufficient.

Safe materialisation.
Compact rights unfold into valid Cardano outputs exactly once. Mechanism: settlement descriptor, min-ADA rules, materialisation plan, and release receipts.

Close/fanout safety.
In V0, no unresolved unilateral virtual claim should survive Head fanout. Mechanism: materialisation-before-fanout rule and Hydra contestation with newer snapshots when stale Head snapshots omit required Lerna state.

No accidental Head-level UTxO explosion.
The factory normally remains compact inside the Head. Mechanism: compact `FactoryState` plus selective materialisation. The protocol deliberately postpones UTxO expansion until local exit, rebalance boundary, or Head shutdown preparation.

Native-asset extension no-drift target.
In a future multi-asset profile, native assets must not appear or disappear merely because a compact proof changes a root. Proposed mechanism: per-policy asset conservation, explicit mint/burn policy checks when relevant, and materialised-output equality under the descriptor. This is an extension target, not part of the ADA-only V0 claim.

22 Failure Scenarios

The following scenarios are part of the V0 specification, not edge cases to be hidden in implementation notes.

Scenario	What happens in V0	Design consequence
Operator refuses to checkpoint	A virtual participant who is not a Head participant cannot force inclusion through Lerna alone	Deployment must name an authorised fallback actor or admit assisted, not unilateral, security
Head participant censors materialisation	If no honest authorised participant can create a newer snapshot, stale Head state may finalise	V0 fails; V1 claim-UTxO or different custody model is required

Scenario	What happens in V0	Design consequence
Watcher is late	The newer Lerna state may be valid but irrelevant to final fanout if the contest deadline passes	Watcher response time is part of the materialisation threshold, not an operational afterthought
Proof material withheld	A signed high-number header without touched leaves, descriptor body, or receipt witnesses cannot be validated	Header signing should bind proof availability policy; otherwise freeze and require repair or full authorisation
Same-number equivocation	Two different headers at the same scope and number do not have an ordering relation	Freeze the affected scope until a strictly higher repair state or full shutdown is signed
Too many outputs	Deterministic materialisation cannot complete with margin	Stop accepting compact opens and enter shutdown preparation before close risk
Receipt replay attempt	A right is presented both as compact state and as a materialised output	Reject because the receipt id must be fresh in the old state, present in the new state, and tied to the replacement output
Native-asset bundle appears in V0-ADA	The factory input or materialised output contains a non-ADA asset	Reject under `ledger_profile = V0-ADA`; multi-asset requires a separate profile and benchmark

23 Limitations and Open Questions

Hydra fanout constraints are central. A Lerna plan that materialises too many outputs too late may fail in practice even if it is theoretically valid. Partial fanout helps but does not remove intrinsic Cardano transaction-size limits, especially for large datums or awkward native-asset bundles.

Head participant limits also matter. Lerna can reduce virtual-channel signing sets, but the outer Head still has a participant set and networking topology. Non-Head users need representation and watcher arrangements.

V1 claim UTxOs are complex. A validator must handle signatures, monotonicity, local exits, non-interference, asset conservation, min-ADA, and proof sizes within Cardano execution budgets. This paper does not claim that such a validator is complete.

Proof size is unknown. Sparse Merkle proofs, KZG-style accumulators, or other commitments may be useful, but the choice must be benchmarked against Aiken or Plutus costs and Head-local transaction ergonomics.

Routing and liquidity discovery are out of scope. Lerna defines factory-local safety, not a payment network. Privacy is mostly out of scope. Commitment roots avoid full plaintext disclosure in factory mode, but access manifests and touched leaves still leak information. Production readiness is not claimed. The immediate output should be a prototype and a test agenda.

24 Evaluation Agenda

The first evaluation should be small and concrete.

24.1 Minimal Hydra Local Demo

Run a local Hydra Head with two or three participants. Create a compact Lerna factory UTxO inside the Head. Execute virtual-channel open, update, local exit, and materialisation transactions. Close and fan out only after materialisation.

24.2 Minimum Prototype Measurements

The first report should include the following table for the ADA-only path. This paper does not invent measurements; it defines the minimum data needed before a stronger claim is credible.

Measurement	Prototype case	Why it matters
Head-local transaction bytes	open, checkpoint, local exit, materialise batch	Determines the safe batch budget
Validator execution units	compact factory spend	Shows whether Aiken/Plutus checks are plausible
Materialised output count per batch	ADA-only exits	Bounds shutdown preparation
Inline datum bytes	compact factory datum and materialised outputs	Detects unfannable large-output risk
Min-ADA locked for materialisation	two-channel local exit	Separates business balance from output viability reserve
Receipt bytes and root growth	repeated local exits	Shows whether receipts become datum bloat
Watcher response time	stale close then contest	Tests the liveness bridge

24.3 Aiken Validator Sketch

Prototype an Aiken validator for the Head-local factory UTxO. It should check domain separation, state-number monotonicity, touched-leaf root updates, release receipts, and a simple ADA-only conservation rule.

24.4 Virtual-channel Update Traces

Generate traces for cooperative update, stale local exit attempt, newer-state materialisation, rebalance between two virtual channels, release receipt creation, and failed non-interference proof.

24.5 Close and Fanout Test Cases

Test Head closure with all virtual channels materialised, a stale Head snapshot missing materialisation followed by contest with a newer snapshot, excessive materialisation output count, and a large inline datum. Native-token output cases belong to the multi-asset extension suite, where rejection under V0-ADA and later multi-asset fanout behaviour can be tested separately.

24.6 Non-interference Tests

Positive tests should show that a local exit touching Alice and Bob does not affect Carol's balance, reserve claim, membership, or exit path. Negative tests should show that a balance-only proof is rejected when shared reserve changes.

24.7 Extension Asset Conservation Matrix

The V0 acceptance path tests lovelace and min-ADA only. The extension suite should later test one native asset, multiple native assets under the same policy, multiple policy IDs, zero-quantity edge cases, and explicit mint/burn boundaries.

25 Conclusion

Lerna Protocol is best understood as a Hydra-native factory discipline. It does not try to replace Hydra's Head lifecycle, does not compete with Hydrozoa, and does not rebrand eltoo for Cardano. Its purpose is narrower and more useful: compactly manage many virtual channels inside Hydra-family Heads while preserving deterministic materialisation, latest-state settlement, asset conservation, and factory-local non-interference.

The most promising first implementation is V0. Keep Lerna entirely inside an open Hydra Head. Use a compact factory UTxO. Require unresolved virtual rights to materialise before Head close and fanout. Add release receipts so compact rights cannot be double-spent after materialisation. Treat V1 L1 claim UTxOs as future research until validators and proof sizes are credible.

If this direction is correct, Lerna is a candidate underexplored layer in the Cardano scaling stack: Hydra gives fast shared eUTxO execution; Hydrozoa and Interhead work broaden the Head-family design space; Lerna supplies a factory-local semantics for making virtual channels safer, compact, and auditable inside that ecosystem.

A Hydra Transaction-flow Trace

This appendix gives the operational V0 trace that a Hydra implementer should expect to reproduce in a local demo. It is intentionally phrased as a sequence of Head-visible actions rather than as an abstract state-machine diagram.

Minimal V0 prototype target. The smallest useful demo is a local Hydra Head with one compact factory UTxO, two ADA-only virtual channels, one local exit, one release receipt, deterministic materialisation before fanout, and a stale snapshot contest scenario. Native assets, dynamic membership, and V1 claim UTxOs should remain outside the first demo. The two-channel case should be chosen so that one right materialises while at least one other right remains compact, since that is the smallest setting that exercises both release receipts and non-interference boundaries.

H1. Open or join a Hydra Head. The Head participant set, Head id, contestation period, and node topology are established by the Hydra workflow. Lerna records the Head id in its factory configuration but does not alter the Head opening protocol.

H2. Deposit funds into the Head. Participants or operators deposit the UTxOs that will fund the Lerna factory reserve. The deposited outputs must already respect Cardano value, datum, reference-script, and min-ADA constraints.

H3. Create the factory UTxO inside the Head. A Head-local transaction creates the compact Lerna factory UTxO, or an equivalent validator-compatible state object for the prototype environment. Its datum binds factoryId, factoryEpoch, the Head id, participant registry root, asset registry root, reserve policy, and settlement descriptor.

H4. Open virtual channels. Head-local transactions, or off-chain states later checkpointed into the Head, create virtual-channel rights inside the factory. Each opening updates the balance, reserve, membership, and subchannel commitments under the access-manifest rules.

H5. Update virtual states. Virtual participants exchange domain-separated latest-state headers. A deployment may checkpoint every update into the Head, or checkpoint only when local exit, rebalance, or liveness pressure requires it.

H6. Request local exit. A participant presents the latest virtual state, access manifest, touched leaves, required signatures, and either a non-interference proof or full factory authorisation.

H7. Materialise Head UTxOs. The Head-local factory validator, or validator-compatible transaction rule, accepts a transaction that consumes the compact factory UTxO, creates ordinary Head UTxOs for the exited rights, writes release receipts into the new compact factory state, and preserves ADA conservation in the first profile.

H8. Prepare for shutdown. If the Head is expected to close, the factory stops accepting new compact rights once the materialisation threshold is crossed. Remaining rights are unfolded by deterministic prefix materialisation until no unresolved unilateral claim remains in the compact state.

H9. Close the Head. A Head participant closes with a Hydra snapshot. If the snapshot is stale with respect to Lerna, an honest watcher must contest with a newer Hydra snapshot containing the latest Lerna checkpoint or materialisation.

H10. Fan out. Hydra fanout distributes ordinary Head UTxOs. In V0, the fanned-out UTxO set should contain materialised Lerna outputs or a cooperatively closed factory state, not unresolved unilateral virtual claims.

B Prototype Validator Checklist

A minimal V0 prototype should report, for each Head-local factory transaction, which checks were enforced by validator or validator-compatible transaction rules, which were enforced by off-chain construction, and which remained test-only assertions.

Validator or transaction-rule enforced.

- factory id and Head id binding;
- factory state-number monotonicity;
- domain-separated signatures over the latest-state header;
- access-manifest hash and transition-family binding;
- touched-root recomputation;
- release receipt creation and consumption;

- ADA-only conservation in the first prototype;
- rejection of non-ADA values under V0-ADA.

Off-chain construction enforced.

- selection of the canonical materialisation prefix;
- min-ADA sufficiency of all materialised outputs;
- transaction-size, datum-size, witness-size, and execution-budget preflight;
- watcher/operator routing to an authorised Head participant;
- operator-budget separation from participant balances.

Test-only until benchmarked.

- reduced-signature non-interference proof; the first prototype may use full-authorisation fallback instead;
- native-asset conservation by policy ID and asset name for extension profiles;
- large datum rejection;
- stale snapshot contest with a newer Lerna checkpoint;
- measured time-to-materialise under the chosen contestation period;
- release-receipt replay attempts across factory epochs and batch indices.

C Open Questions

Q1. How should virtual participants who are not Head participants delegate watching and contest authority without turning operators into custodians?

Q2. What realistic materialisation bound can be achieved under ordinary Hydra contestation windows for ADA-only outputs, then for native-asset bundles?

Q3. Which commitment scheme offers the best proof-size and validator-cost trade-off for touched leaves and non-interference proofs?

Q4. Can a V1 L1 claim UTxO enforce state monotonicity, signatures, non-interference, min-ADA, and native-asset conservation within practical Aiken/Plutus budgets?

Q5. Would Hydrozoa-style dynamic membership change the assumptions behind the factory participant registry and rights-dependency graph?

Q6. How should release receipts be encoded so that they prevent double claims without causing datum bloat in long-lived factories?

Q7. What is the minimal V0 factory datum that remains auditable without making Head-local transactions unwieldy?

References

[1] S. Chakravarty, M. D. Coretti, M. Fitzi, P. Gazi, P. Kant, A. Kiayias, and A. Russell. Hydra: Fast Isomorphic State Channels. IACR Cryptology ePrint Archive, Report 2020/299. https://eprint.iacr.org/2020/299.pdf.

[2] Cardano Scaling. Hydra Head protocol documentation. https://hydra.family/head-protocol/docs/protocol-overview.

[3] Cardano Scaling. Hydra implementation repository. https://github.com/cardano-scaling/hydra.

[4] Cardano Scaling. Hydra known issues and limitations. Version 2.0.0 documentation, consulted 5 June 2026. https://hydra.family/head-protocol/docs/known-issues.

[5] Obsidian Systems. Hydra Pay repository. https://github.com/cardano-scaling/hydra-pay.

[6] Cardano Hydrozoa. Hydrozoa repository and protocol materials; local repository marks the project as pre-alpha and under active development. https://github.com/cardano-hydrozoa/hydrozoa.

[7] G. Flerovsky, I. Rodionov, and P. Dragos. Hydrozoa: Lightweight state channels for Cardano. Technical specification working draft, 7 November 2025. https://cardano-hydrozoa.github.io/hydrozoa/hydrozoa.pdf.

[8] M. Jourenko, M. Larangeira, and K. Tanaka. Interhead Hydra: Two Heads are Better than One. IACR Cryptology ePrint Archive, Report 2021/1188. https://eprint.iacr.org/2021/1188.pdf.

[9] Aiken. A modern smart contract platform for Cardano. https://github.com/aiken-lang/aiken.

[10] Cardano Documentation. The Extended UTxO accounting model. https://docs.cardano.org/about-cardano/learn/eutxo-explainer/.

[11] C. Decker, R. Russell, and O. Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. https://blockstream.com/eltoo.pdf.

[12] C. Burchert, C. Decker, and R. Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file/
/49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.

[13] S. Dziembowski, S. Faust, and K. Hostakova. General State Channel Networks. ACM CCS 2018.